Greedy Algorithms

A short list of categories

- Algorithm types:
  - Simple recursive algorithms
  - Backtracking algorithms
  - Divide and conquer algorithms
  - Dynamic programming algorithms
  - Greedy algorithms
  - Branch and bound algorithms
  - Brute force algorithms
  - Randomized algorithms

Optimization problems



- An optimization problem is one in which you want to find, not just *a* solution, but the *best* solution
- A "greedy algorithm" sometimes works well for optimization problems
- A greedy algorithm works in phases. At each phase:
  - You take the best you can get right now, without regard for future consequences
  - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

The Greedy Method Technique

- **The greedy method** is a general algorithm design paradigm, built on the following elements:
  - ***configurations***: *different choices, collections, or values to find*
  - ***objective function***: *a score assigned to configurations, which we want to either maximize or minimize*
- It works best when applied to problems with the **greedy-choice** property:
  - *a globally-optimal solution can always be found by a series of local improvements from a starting configuration.*

Making a Greedy Choice

- **Greedy choice:**
  - *We make the choice that looks best at the moment.* (not looking at the future at all)

i.e. every time we make a choice, we greedily try to maximize our profit.

General Form of Greedy

IDEA: Geedy algorithm locally optimal choice hope will lead to a globally optimal solution.

```
SolType Greedy(Type a[], int n)
{
        SolType solution = EMPTY;
        for(int i=1; i<=n; i++)
        {
                Type x = Select(a);
                if Feasible(solution, x);
                        solution = Union(solution, x);
        }
        return solution;
}
```

Example: Counting money

- Suppose you want to count out a certain amount of money, using the fewest possible bills and coins
- A greedy algorithm would do this would be:
  At each step, take the largest possible bill or coin that does not overshoot
  - Example: To make $6.39, you can choose:
    - a $5 bill
    - a $1 bill, to make $6
    - a 25¢ coin, to make $6.25
    - A 10¢ coin, to make $6.35
    - four 1¢ coins, to make $6.39
- For US money, the greedy algorithm always gives the optimum solution

A failure of the greedy algorithm

- In some (fictional) monetary system, “krons” come in 1 kron, 7 kron, and 10 kron coins
- Using a greedy algorithm to count out 15 krons, you would get
  - A 10 kron piece
  - Five 1 kron pieces, for a total of 15 krons
  - This requires six coins
- A better solution would be to use two 7 kron pieces and one 1 kron piece
  - This only requires three coins
- The greedy algorithm results in a solution, but not in an optimal solution

Greedy Algorithms

- A **greedy algorithm** makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
  The choice made at each step must be:
  - ***Feasible***
    - Satisfy the problem's constraints
  - ***locally optimal***
    - Be the best local choice among all feasible choices
  - ***Irrevocable***
    - Once made, the choice can't be changed on subsequent steps.

- Do greedy algorithms always yield optimal solutions?

Applications of the Greedy Strategy

Optimal solutions:	Approximations:
• *change making* • *Minimum Spanning Tree (MST)* • *Single-source shortest paths* • *Huffman codes*	• *Traveling Salesman Problem (TSP)* • *Knapsack problem* • *Many other optimization problems*

Making Change

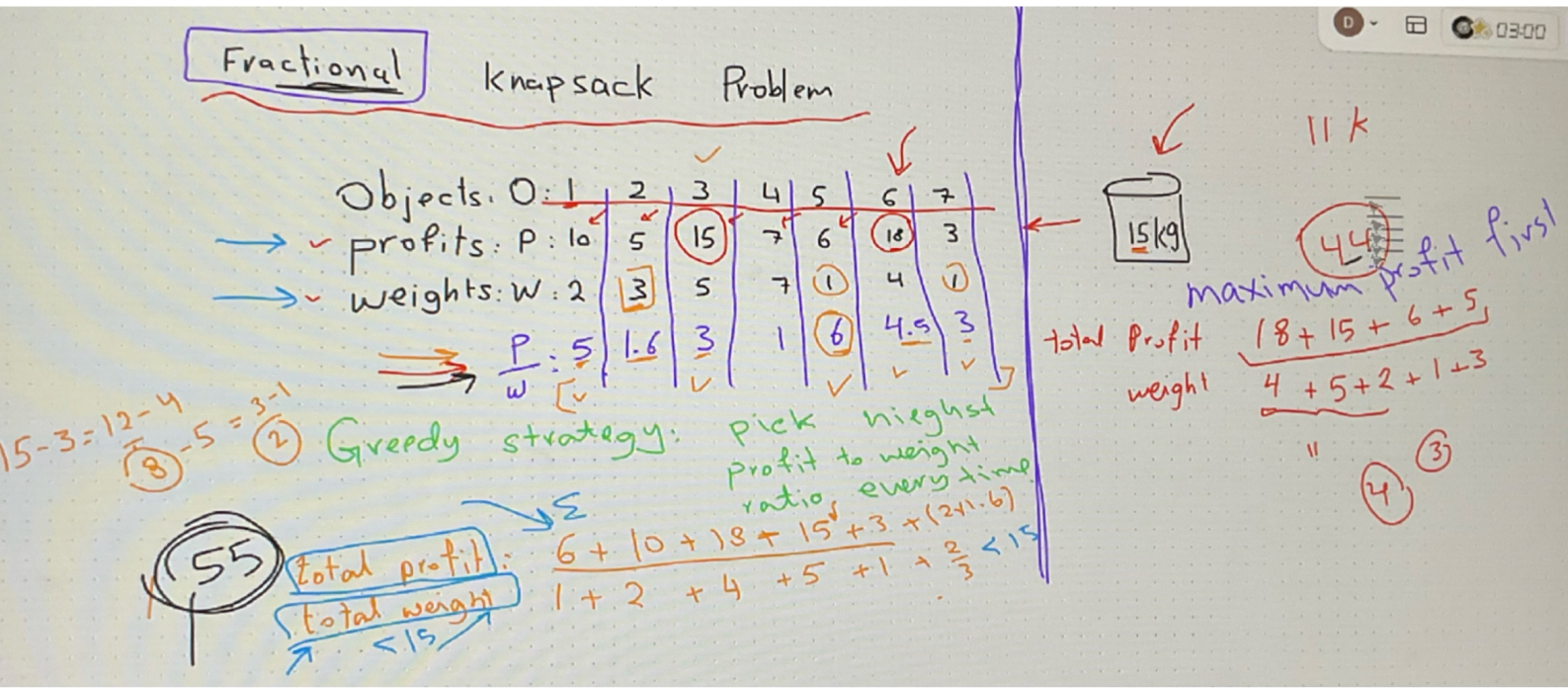
- **Problem**: A dollar amount to reach and a collection of coin amounts to use to get there.

- **Configuration:** A dollar amount yet to return to a customer plus the coins already returned
- **Objective function**: Minimize number of coins returned.
- **Greedy solution**: Always return the largest coin you can

- **Example 1**: Coins are valued $.32, $.08, $.01
  - *Has the greedy-choice property, since no amount over $.32 can be made with a minimum number of coins by omitting a $.32 coin (similarly for amounts over $.08, but under $.32).*
- **Example 2**: Coins are valued $.30, $.20, $.05, $.01
  - *Does not have greedy-choice property, since $.40 is best made with two $.20's, but the greedy solution will pick three coins (which ones?)*

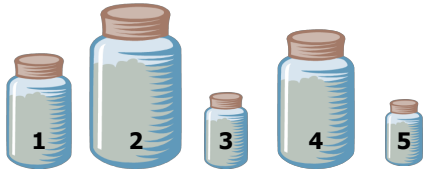
The Fractional Knapsack Problem

- **Given**: A set S of n items, with each item i having
  - b_i - *a positive benefit*
  - w_i - *a positive weight*
- **Goal**: Choose items with maximum total benefit but with weight at most W.
- If we are allowed to take fractional amounts, then this is the **fractional knapsack problem**.
  - *In this case, we let x_i denote the amount we take of item i*
  - *Objective: maximize* $\sum_{i \in S} b_i x_i$
  - *Constraint:* $\sum_{i \in S} w_i x_i \leq W$


Fractional knapsack Problem
Objects: O: 1 2 3 4 5 6 7
profits: P: 10 5 15 7 6 18 3
weights: W: 2 3 5 7 1 4 1
P/w: 5 1.6 3 1 6 4.5 3
Greedy strategy: pick highest profit to weight ratio every time
6 + 10 + 18 + 15 + 3 + (2+1.6)
1 + 2 + 4 + 5 + 1 + 2/3 < 15
total profit
total weight < 15
55
15 kg
11 k
44
maximum profit first
total Profit 18 + 15 + 6 + 5
weight 4 + 5 + 2 + 1 + 3
15-3=12-4
8-5=3-1
2
3
4

Example

- **Given**: A set S of n items, with each item i having
  - b_i - *a positive benefit*
  - w_i - *a positive weight*
- **Goal**: Choose items with maximum total benefit but with weight at most W.



	1	2	3	4	5
Weight:	4 ml	8 ml	2 ml	6 ml	1 ml
Benefit:	$12	$32	$40	$30	$50
Value: ($ per ml)	3	4	20	5	50



10 ml

"knapsack"

Solution:
- 1 ml of 5
- 2 ml of 3
- 6 ml of 4
- 1 ml of 2

The Fractional Knapsack Algorithm

- Greedy choice: Keep taking item with highest **value** (benefit to weight ratio)
  - *Run time: O(n log n). Why?*
- Correctness: Suppose there is a better solution
  - *there is an item i with higher value than a chosen item j (i.e., $v_i > v_j$) but $x_i < w_i$ and $x_j > 0$ If we substitute some i with j, we get a better solution*
  - *How much of i: min{$w_i - x_i$, x_j}*
  - *Thus, there is no better solution than the greedy one*

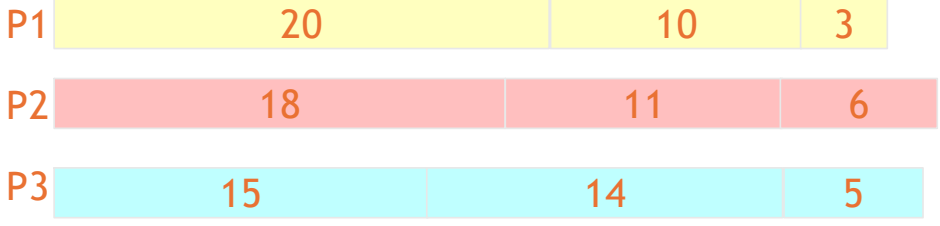
Algorithm ***fractionalKnapsack*****(*S*, *W*)**
Input: set ***S*** of items w/ benefit b_i and weight w_i; max. weight M
Output: amount x_i of each item i to maximize benefit with weight at most M
for ***each item i in S***
  $x_i \leftarrow 0$
  $v_i \leftarrow b_i / w_i$ {value}
Sort objects based on their values. n log n
Current_weight ← M
for (i=1; i<= n; i++) {
  if $w_i >$ ***Current_weight*** break;
  $x_i \leftarrow 1.0$
  current_weight =**current_weight – w_i**
  }
  If (i<= n) x_i = Current_weight / w_i
}

n

Cnlogn + n
↑
dominating term
so O(nlogn)

A scheduling problem

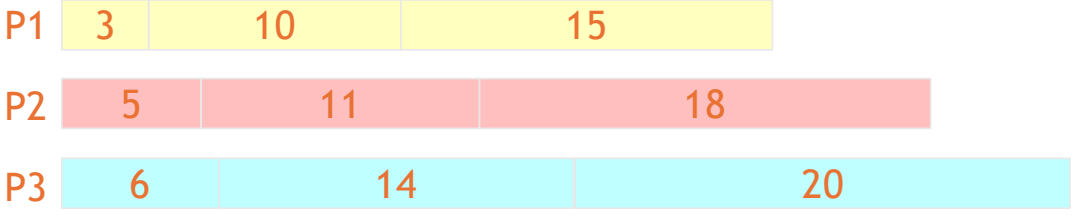
- You have to run nine jobs, with running times of 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes
- You have three processors on which you can run these jobs
- You decide to do the longest-running jobs first, on whatever processor is available



- Time to completion:
  - 18 + 11 + 6 = 35 minutes
- This solution isn't bad, but we might be able to do better

Another approach

- What would be the result if you ran the *shortest* job first?
- Again, the running times are 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes



- That wasn't such a good idea; time to completion is now 6 + 14 + 20 = 40 minutes
- Note, however, that the greedy algorithm itself is fast
  - All we had to do at each stage was pick the minimum or maximum

You have to run nine jobs, with running times of 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes
You have three processors on which you can run these jobs

one possible * Greedy strategy: is to assign largest task first (works)

Apply your strategy

P1: 20 + 10 + 3 = 33
P2: 18 + 11 + 5 = 34
P3: 15 + 14 + 6 = 35

better

---

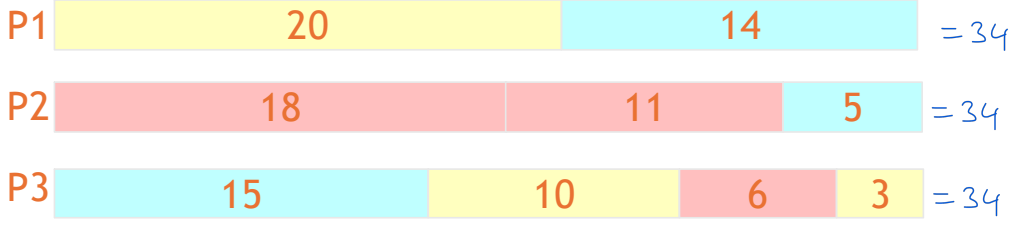
another greedy strategy: is to assign smallest task first

P1: 3 + 10 + 15 = 28 ✗
P2: 5 + 11 + 18 = 34
P3: 6 + 14 + 20 = 40

worst

An optimum solution

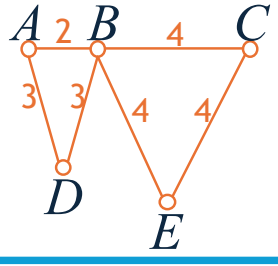
- Better solutions do exist:



  - This solution is clearly optimal (why?)
  - How do we find such a solution?
    - One way: Try all possible assignments of jobs to processors (brute force)
    - Unfortunately, this approach can take exponential time

Traveling salesman

- A salesman must visit every city (starting from city ***A***), and wants to cover the least possible distance
  - He can revisit a city (and reuse a road) if necessary
- He does this by using a greedy algorithm: He goes to the next nearest city from wherever he is (shortest path)



- From ***A*** he goes to ***B***
- From ***B*** he goes to ***D***
- This is *not* going to result in a shortest path!
- The best result he can get now will be ***ABDBCE***, at a cost of 16 (sub-optimal)
- An actual least-cost path from ***A*** is ***ADBCE***, at a cost of 14

Other greedy algorithms

- **Dijkstra's algorithm** for finding the shortest path in a graph
  - Always takes the *shortest* edge connecting a known node to an unknown node
- **Kruskal's algorithm** for finding a minimum-cost spanning tree
  - Always tries the *lowest-cost* remaining edge
- **Prim's algorithm** for finding a minimum-cost spanning tree
  - Always takes the *lowest-cost* edge between nodes in the spanning tree and nodes not yet in the spanning tree

Scheduling with Deadlines (was on a final last semester)

- Problem: Determine a schedule with maximum total profit, given that each job has a profit that will be obtained only if the job is scheduled by its deadline.
- Input
  - *n, number of jobs*
  - *Profit for each job*
  - *Deadline for each job*
- Output: an optimal sequence for the jobs
- Assumption: each job takes one unit of time to finish

Job	Deadline	Profit
1	2	30
2	1	35
3	2	25
4	1	40

Feasible Schedule	Total Profit
[1,3]	30+25=55
[2,1]	35+30=65
[2,3]	35+25=60
[3,1]	25+30=55
[4,1]	40+30=70
[4,3]	40+25=65

Scheduling with Deadlines
Greedy Method

- Steps:
  1. *First sort the jobs in nonincreasing order by profit*
  2. *Check job in sequence and add it to the schedule if it is feasible*
  3. *Add the new job to the sequence according to nondecreasing order of the deadlines*
- Feasible sequence: if all the jobs in the sequence start by their deadline
  - *[4,1] is a feasible sequence, but [1,4] is not*
- Feasible set: if there exists at least one feasible sequence for the jobs in the set
  - *{1,4} is a feasible set, but {2,4} is not*
- In the previous example, we call [4,1] as an optimal sequence and the set of the jobs {1,4} as an optimal set

Scheduling with Deadlines Greedy Method, Example

- The jobs are sorted by their profits
- The Greedy algorithm does the following:

1. S = empty
2. S={1} because the sequence [1] is feasible
3. S={1,2} because the sequence [2,1] is feasible
4. {1,2,3} rejected because there is no feasible sequence
5. S={1,2,4} because the sequence [2,1,4] is feasible
6. {1,2,4,5} rejected because there is no feasible sequence
7. {1,2,4,6} rejected because there is no feasible sequence
8. {1,2,4,7} rejected because there is no feasible sequence

Job	Deadline	Profit
1	3	40
2	1	35
3	1	30
4	3	25
5	1	20
6	3	15
7	2	10